

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME	Artificial Intelligence
COURSE CODE	CSA17
COURSE OUTCOME COVERED	CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)
ASSESSMENT TOOL WEIGHTAGE	50%
DURATION & TOTAL MARKS	1 Hour 50 Marks
TOTAL QUESTIONS	5 (Scenario Based)

Code of Conduct

I, **B. Vishnu / 192512410**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **B. Vishnu**

Annexure A - Scenario Based Assignment Solutions

1. Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic...

i. Greedy Best-First Search Behavior: Greedy Best-First Search evaluates nodes based strictly on the heuristic $h(n)$ (straight-line distance to the target). Its *strength* lies in its speed, as it aggressively moves toward the goal, which is ideal in extreme urgency. However, its *weakness* is that it ignores actual path constraints (weather, terrain, blocked roads). Under uncertainty, it may fly the drone into a severe storm or a blocked path just because it seemed geographically closer, leading to potential failure or massive backtracking delays.

ii. A* Handling the Situation: A* search balances the cost by evaluating $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost incurred (terrain and weather penalties) and $h(n)$ is the heuristic. This allows the drone to find an optimal path that is both short and safe, avoiding paths that look close but have prohibitively high movement costs due to bad weather.

iii. Impact of Heuristic Accuracy: If the heuristic accurately reflects the true remaining distance without overestimating, A* is guaranteed to be optimal. If the heuristic overestimates (e.g., ignores tailwinds), A* may become suboptimal. For Greedy search, a poor heuristic will easily lead the drone into dead ends, significantly increasing delivery time.

iv. Impact of Dynamic Changes: Standard static search algorithms fail when the environment changes after the path is calculated. Blocked paths discovered in real-time require the algorithm to recalculate the path, which can be computationally expensive if done from scratch every time an obstacle is encountered.

v. Recommendation: The most suitable algorithm is **D* Lite (Dynamic A*)**. Real-time constraints and dynamic changes (blocked paths) make static A* inefficient. D* Lite uses the A* principles but dynamically updates the path based on new information without recalculating from scratch, perfectly balancing optimality, adaptability, and safety.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections...

i. Problem Formulation & Inefficiency of Traditional Search: **Formulation:** *State:* A specific configuration of traffic signal timings across all intersections. *Objective Function:* Minimize total waiting time, fuel consumption, and congestion. **Inefficiency:** Traditional search (like A* or BFS) is inefficient because the state space (all possible continuous timings for hundreds of signals) is too massive. It is impossible to systematically explore the entire tree in real-time.

ii. Hill Climbing: Hill Climbing starts with a random set of signal timings and iteratively tweaks the timings of individual signals to find a neighboring state with a better objective score (less congestion). *Limitation:* It easily gets stuck in a local maximum (or minimum for costs)—a configuration that works well for a few intersections but causes gridlock elsewhere, and no single tweak can improve it.

iii. Issues in Real-World Context: *Local Maxima:* A state where traffic is flowing well in one district, but globally suboptimal. *Plateaus:* Changing a green light by 1 second doesn't change the traffic flow at all, leaving the algorithm wandering without clear direction. *Ridges:* Requiring simultaneous changes at multiple adjacent intersections to see improvement; changing them one at a time yields worse results.

iv. Overcoming Limitations: *Simulated Annealing* overcomes this by probabilistically allowing "bad" timing tweaks initially (high temperature) to escape local maxima, slowly cooling down to settle on the global optimum. *Genetic Algorithms (GA)* overcome it by maintaining a population of different timing strategies, combining (crossover) the best aspects of different city zones, and introducing random mutations to continuously explore new timing patterns.

v. Recommendation: **Genetic Algorithms (GA)** are the most suitable. They scale incredibly well to large state spaces (hundreds of intersections) and can continuously adapt to dynamically changing traffic patterns by maintaining a diverse population of solutions, avoiding the pitfalls of single-state local search methods.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain...

i. Online vs Offline Search: Offline search requires a complete, known map in advance to calculate the entire path before taking the first step. Since the Mars terrain is unknown, this is impossible. The rover must use an *online search agent*, which interleaves computation and action—it takes a step, observes the new environment, and then decides the next step based on updated sensor data.

ii. Challenges: *Partial Observability:* The rover's sensors have limited range, meaning it might commit to a path that eventually leads to a massive uncrossable crater. *Dynamic Environments:* Sandstorms might alter the terrain or obscure vision, requiring the rover to constantly adapt to states that change outside of its control.

iii. Internal Model Updating: As the rover moves, it uses techniques like SLAM (Simultaneous Localization and Mapping). It updates its internal grid, marking nodes as "visited," mapping exact costs of traversed terrain, and noting the locations of newly discovered hazards, subsequently updating heuristic estimates for nearby nodes.

iv. Uninformed vs Informed Online Search: Uninformed online search (like a random walk) is highly inefficient and dangerous, often leading to endless wandering or falling into traps. Informed online search utilizes heuristic functions (e.g., estimated distance to a target zone) to purposely guide exploration, avoiding states that lead away from the objective while mapping.

v. Recommendation: Learning Real-Time A* (LRTA*) is the best approach. It allows the rover to move safely in real-time by relying on heuristics, and it constantly updates those heuristics as it discovers the true cost of the terrain. This prevents the rover from getting stuck in dead ends, ensuring adaptability and safety.

4. Scenario: A university is designing an AI system to automatically generate exam timetables...

i. Problem Formulation (CSP): *Variables:* The set of all exams/courses to be scheduled (e.g., Math, Physics). *Domains:* The available time slots and exam halls for each variable (e.g., [Monday 9AM Room A, Tuesday 2PM Room B]). *Constraints:* - *Binary constraints:* No student can take two exams at the same time. - *Unary constraints:* Subject X must be before Subject Y; Hall capacity must meet student count.

ii. Backtracking Search: The algorithm picks a variable (exam) and assigns it a valid value (time/hall). It then proceeds to the next exam. If it reaches an exam where no valid time slot exists without violating constraints, it "backtracks" to the previous exam, unassigns it, and tries a different slot.

iii. Constraint Propagation (Forward Checking): Forward checking significantly improves efficiency. When an exam is assigned to a specific time slot, the system instantly removes that slot from the domains of all other exams that share students with the assigned exam. This catches inevitable failures immediately, drastically reducing the search tree depth.

iv. Scalability Strategy: To handle growth, the system should use a combination of **Minimum Remaining Values (MRV) heuristic** and **Degree Heuristic**. MRV schedules the most restricted exams first (those with the fewest possible valid slots). Combined with Forward Checking and Conflict-Directed Backjumping (jumping back to the actual variable that caused the conflict, rather than just the immediate previous one), the system can scale efficiently to thousands of students.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)...

i. Minimax Modeling: Minimax models the real-time strategy game as a search tree. The AI represents the "MAX" player, trying to choose moves that maximize its utility score, while assuming the human opponent is the "MIN" player, making moves that minimize the AI's score. It explores alternative futures up to a certain depth and propagates the best guaranteed scores back to the root.

ii. Computational Challenges: An RTS game has a massive branching factor b (hundreds of possible actions per second) and an indefinite depth d . Exploring the full tree requires $O(b^d)$ time complexity. Computing this within a fraction of a second is impossible, leading to a system freeze or delayed decisions.

iii. Alpha-Beta Pruning Efficiency: Alpha-Beta pruning tracks the highest score MAX is guaranteed (α) and the lowest score MIN is guaranteed (β). If a branch is found that proves a move is worse than a previously examined move, the algorithm stops evaluating that branch (prunes it). This can reduce the time complexity to $O(b^{d/2})$ in the best case, allowing the AI to search twice as deep in the same amount of time.

iv. Evaluation Function Design: Because the AI cannot search to the end of the game, it must evaluate non-terminal states.

Function: $Utility(s) = w_1(Military\ Units) + w_2(Resource\ Income) + w_3(Map\ Control) + w_4(Base\ Health)$.

Justification: These factors represent the core economic and military advantage in an RTS. The weights (w) adjust based on the game phase (e.g., resources are prioritized early game, military late game).

v. Recommendation for Balancing: Implement **Iterative Deepening with Alpha-Beta Pruning**. The AI searches to depth 1, then depth 2, then depth 3, and so on. When the strict time limit expires, it simply halts the search and returns the best move found in the deepest completed iteration. This ensures an optimal-as-possible decision is always ready instantly.
